

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ

РГУ им. А.Н. Косыгина

(Технологии. Дизайн. Искусство)

Институт Информационных технологий и цифровой
трансформации

Кафедра «Искусственного интеллекта, прикладной математики и
программирования»

Курсовая работа
по дисциплине «Вероятностное моделирование процессов и
систем»

на тему: Создание приложений вероятностного моделирования

Выполнил: студент группы ИТПМ-124

Фриев Д.С.

(Фамилия, имя, отчество)

(подпись)

Принял: доцент кафедры ИИПМиП

Романенков А.М.

(Фамилия, имя, отчество)

(подпись)

Оценка:

Дата:

Москва, 2026

Содержание

Введение	3
0.1 Цель и задачи	3
0.2 Используемые технологии	3
0.3 Структура пояснительной записки	4
1 Выбор символа из алфавита	5
1.1 Аналитическое решение	5
1.2 Эмпирическая проверка	6
2 Моделирование эпидемии на графе социальных контактов	9
2.1 Модель и структуры данных	9
2.2 Ядро симуляции	9
2.3 Аналитические запросы к результатам	10
2.4 Тестовые данные и визуализация	11
3 Распространение слухов	13
3.1 Модель и алгоритм	13
3.2 Ядро симуляции	13
3.3 Визуализация и результаты	14
4 Моделирование движения материальной точки со случайным шагом	16
5 Кластеры и k-паттерны	16
6 Ступенчатая фигура	16
7 Комплексное блуждание	16
8 Бочка с бензином	16
9 Криптоанализ	16
10 Подбрасывание монеты из кошелька	16
11 Дискретная случайная величина	16
12 Игла Бюффона	16
13 Статистика графа	17

Введение

0.1 Цель и задачи

Цель работы — программная реализация комплекса задач вероятностного моделирования. Для достижения цели необходимо решить следующие задачи:

- разработать математические модели для каждого задания;
- продумать и создать архитектуру приложений;
- реализовать пользовательский интерфейс для управления параметрами моделирования;

0.2 Используемые технологии

Программная реализация выполнена на языке C++ стандарта C++23 с использованием следующих библиотек и инструментов:

- CMake (<https://github.com/Kitware/CMake>) — система сборки проекта;
- Qt6 Quick (<https://github.com/qt/qtdeclarative>) — фреймворк для разработки графического интерфейса;
- GLFW (<https://github.com/glfw/glfw>), Dear ImGui (<https://github.com/ocornut/imgui>) и ImPlot (<https://github.com/epezent/implot>) — для визуализации графиков дискретной случайной величины в одном из заданий;
- nlohmann::json (<https://github.com/nlohmann/json>) — парсинг конфигурационных файлов;
- kursach-autogen (<https://github.com/koftamainee/kursach-autogen>) — генерация пояснительной записки.

Все разработанные приложения устойчивы к аварийным завершениям, предусмотрена обработка ошибок ввода-вывода и некорректных параметров.

0.3 Структура пояснительной записки

Документ имеет следующую структуру:

- введение;
- описание задач с подробностями их реализации;
- выводы;
- список использованных источников;
- приложение со ссылкой на репозиторий исходного кода.

1 Выбор символа из алфавита

Из упорядоченного алфавита, содержащего N различных символов, последовательно выбирают два символа: сначала один, затем — один из оставшихся. Все исходы считаются равновероятными. Требуется определить вероятность того, что в первый раз, во второй раз и оба раза будет выбран символ с чётным номером. Индексация элементов алфавита начинается с 1. Рассматриваются два варианта нумерации после первой выборки: а) индексы оставшихся элементов не изменяются; б) индексы изменяются (переиндексируются начиная с 1). Необходимо реализовать приложение для проведения эксперимента, которое принимает через командную строку мощность алфавита N и количество экспериментов C , и выполнить программное сравнение эмпирической и аналитической вероятностей.

1.1 Аналитическое решение

Пусть алфавит содержит N символов с номерами от 1 до N . Число чётных номеров равно $\lfloor N/2 \rfloor$. Первый символ выбирается равновероятно из всех N элементов.

Случай а) — фиксированные индексы.

$$P(\text{первый чётный}) = \frac{\lfloor N/2 \rfloor}{N} \quad (1)$$

При фиксированных индексах после удаления первого элемента оставшиеся $N - 1$ символов сохраняют свои номера. Вероятность того, что второй выбранный символ имеет чётный номер, зависит от чётности первого:

$$P(\text{второй чётный}) = \frac{\lfloor N/2 \rfloor}{N} \cdot \frac{\lfloor N/2 \rfloor - 1}{N - 1} + \frac{\lceil N/2 \rceil}{N} \cdot \frac{\lfloor N/2 \rfloor}{N - 1} \quad (2)$$

$$P(\text{оба чётных}) = \frac{\lfloor N/2 \rfloor}{N} \cdot \frac{\lfloor N/2 \rfloor - 1}{N - 1} \quad (3)$$

Случай б) — переиндексация после первой выборки.

После удаления первого элемента оставшиеся $N - 1$ символов получают новые порядковые номера от 1 до $N - 1$. Число чётных позиций среди $N - 1$ элементов равно $\lfloor (N - 1)/2 \rfloor$. Вероятность чётного номера при первом выборе не изменяется:

$$P(\text{первый чётный}) = \frac{\lfloor N/2 \rfloor}{N} \quad (4)$$

После переиндексации вероятность чётного номера при втором выборе не зависит от результата первого, поскольку новая нумерация определяется позицией в оставшемся списке:

$$P(\text{второй чётный}) = \frac{\lfloor (N-1)/2 \rfloor}{N-1} \quad (5)$$

$$P(\text{оба чётных}) = \frac{\lfloor N/2 \rfloor}{N} \cdot \frac{\lfloor (N-1)/2 \rfloor}{N-1} \quad (6)$$

1.2 Эмпирическая проверка

Программа принимает два параметра командной строки: мощность алфавита N и количество экспериментов C . Каждый эксперимент моделирует два последовательных выбора символа. Для генерации случайных чисел используется Вихрь Мерсенна (`std::mt19937`). Запуск серии экспериментов выполняется через вспомогательный класс `TaskRunner`, который принимает произвольный эксперимент в виде callable и возвращает вектор результатов.

Listing 1: Эксперимент с фиксированными индексами

```

1 auto experiment_fixed_indices = [alphabet_size](auto& rng) {
2     std::uniform_int_distribution<size_t> dist1(1, alphabet_size);
3     size_t first = dist1(rng);
4
5     std::vector<size_t> remaining;
6     remaining.reserve(alphabet_size - 1);
7     for (size_t i = 1; i <= alphabet_size; i++) {
8         if (i != first) remaining.push_back(i);
9     }
10
11     std::uniform_int_distribution<size_t> dist2(0, remaining.size() -
12         1);
13     size_t second = remaining[dist2(rng)];
14
15     return Outcome{first % 2 == 0, second % 2 == 0};
16 };

```

Listing 2: Эксперимент с переиндексацией после первого выбора

```

1 auto experiment_reindexed = [alphabet_size](auto& rng) {
2     std::vector<size_t> indices(alphabet_size);
3     std::iota(indices.begin(), indices.end(), 1);
4
5     std::uniform_int_distribution<size_t> dist1(0, indices.size() -
6         1);
7     size_t first_pos = dist1(rng);

```

```

7   bool first_even = (first_pos + 1) % 2 == 0;
8
9   indices.erase(indices.begin() + first_pos);
10
11  std::uniform_int_distribution<size_t> dist2(0, indices.size() -
12      1);
13  size_t second_pos = dist2(rng);
14  bool second_even = (second_pos + 1) % 2 == 0;
15
16  return Outcome{first_even, second_even};
};

```

Для подтверждения корректности полученных аналитических выражений был проведён вычислительный эксперимент при фиксированном размере алфавита $N = 100$ и количестве экспериментов $C = 10^8$ (сто миллионов итераций). Такой объём выборки обеспечивает высокую точность оценки вероятностей и позволяет надёжно проверить сходимость эмпирических частот к теоретическим значениям.

Результаты эксперимента представлены на рис. 1.

```

target/tasks/01-alphabet-sampling on ʘ master [!?]
> ./alphabet-sampling 100 100000000
Running fixed experiment...
[=====] 100%
Running reindexed experiment...
[=====] 100%

Fixed indices experiment
First even:  0.499913
Second even: 0.500032
Both even:   0.247482

Reindexed after first draw experiment
First even:  0.499995
Second even: 0.494859
Both even:   0.247438

```

Рис. 1. Результаты эксперимента при $N = 100$, $C = 10^8$

Полученные эмпирические вероятности:

Таблица 1. Результаты эксперимента при $N = 100$, $C = 10^8$

Вариант нумерации	Первый чётный	Второй чётный	Оба чётных
Фиксированные индексы	0.499913	0.500032	0.247482
Переиндексация	0.499995	0.494859	0.247438

Теоретические значения при $N = 100$ составляют:

$$P_{\text{first}} = \frac{50}{100} = 0.5$$

$$P_{\text{second}}^{(a)} = 0.5$$

$$P_{\text{second}}^{(b)} = \frac{49}{99} \approx 0.494949$$

$$P_{\text{both}} = \frac{50}{100} \cdot \frac{49}{99} \approx 0.247475$$

Сравнение эмпирических и теоретических значений показывает высокую точность моделирования. Максимальное абсолютное отклонение составляет 8.7×10^{-5} для случая фиксированных индексов и 9.0×10^{-5} для случая переиндексации. Таким образом, экспериментальные данные подтверждают корректность аналитических формул и правильность реализации обоих вариантов нумерации. Отметим также, что вероятность P_{both} совпадает для обоих вариантов нумерации (отличие составляет 4.4×10^{-5}), что следует из комбинаторного смысла: выбор двух чётных элементов из множества не зависит от последующей перенумерации.

2 Моделирование эпидемии на графе социальных контактов

Требуется реализовать стохастическую модель распространения инфекции контактным путём на неориентированном графе знакомств. Для каждого контакта «здоровый–инфицированный» заражение происходит с вероятностью $p_1 \in (0; 1]$. Инфицированный выздоравливает на каждом шаге с вероятностью $p_2 \in [0; 1]$. Программа должна предоставлять загрузку графа, пошаговое моделирование с визуализацией на графе и аналитические запросы к результатам.

2.1 Модель и структуры данных

Граф контактов хранится в виде списка смежности. Каждая вершина — структура **Person** с полями: идентификатор, состояние (Healthy, Infected, Recovered) и вектор идентификаторов соседей.

Состояния образуют конечный автомат: Healthy $\rightarrow$ Infected (с вероятностью p_1 при контакте), Infected $\rightarrow$ Recovered (с вероятностью p_2 на каждом шаге). Состояние Recovered — поглощающее. Повторное заражение невозможно.

Listing 3: Структура вершины и состояния эпидемиологической модели

```
1 enum class PersonState { Healthy, Infected, Recovered };
2
3 struct Person {
4     int id;
5     PersonState state = PersonState::Healthy;
6     std::vector<int> contacts;
7 };
```

Генератор псевдослучайных чисел — Вихрь Мерсенна (`std::mt19937`) с равномерным распределением на $[0, 1)$. На каждом шаге для всех инфицированных перебираются соседи, и заражение происходит при $U(0, 1) < p_1$. Затем аналогично проверяется выздоровление.

2.2 Ядро симуляции

Listing 4: Метод `step()` — ядро симуляции

```
1 const std::vector<Person>& Simulation::step() {
2     if (m_current_step == 0) {
3         std::uniform_int_distribution<size_t> pick(0, m_people.size()
4             - 1);
5         m_people[pick(m_rng)].state = PersonState::Infected;
```

```

5         m_current_step++;
6         return m_people;
7     }
8
9     std::vector<int> newly_infected;
10    for (const auto& person : m_people) {
11        if (person.state != PersonState::Infected) continue;
12        for (int cid : person.contacts) {
13            if ((m_people[cid].state == PersonState::Recovered ||
14                m_people[cid].state == PersonState::Healthy) &&
15                m_dist(m_rng) < p_infect)
16                newly_infected.push_back(cid);
17        }
18    }
19    for (int id : newly_infected) m_people[id].state = PersonState::
        Infected;
20
21    for (auto& person : m_people)
22        if (person.state == PersonState::Infected && m_dist(m_rng) <
            p_recover)
23            person.state = PersonState::Recovered;
24
25    m_current_step++;
26    return m_people;
27 }

```

2.3 Аналитические запросы к результатам

Реализованы четыре поисковых запроса, выполняемых за один проход по массиву вершин:

- Все не заразившиеся — вершины, оставшиеся в состоянии Healthy.
- Все исцелившиеся — вершины в состоянии Recovered.
- Исцелившиеся, чьё окружение не исцелилось — Recovered-вершины, среди соседей которых есть хотя бы одна не-Recovered.
- Не заразившиеся, чьё окружение полностью заражено — Healthy-вершины, у которых все соседи имеют состояние Infected.

Listing 5: Поиск исцелившихся с неисцелившимся окружением

```

1 std::vector<int> Simulation::recovered_with_sick_contacts() const {
2     std::vector<int> r;

```

```

3   for (const auto& p : m_people) {
4       if (p.state != PersonState::Recovered) continue;
5       for (int cid : p.contacts) {
6           if (m_people[cid].state != PersonState::Recovered) {
7               r.push_back(p.id);
8               break;
9           }
10      }
11  }
12  return r;
13 }

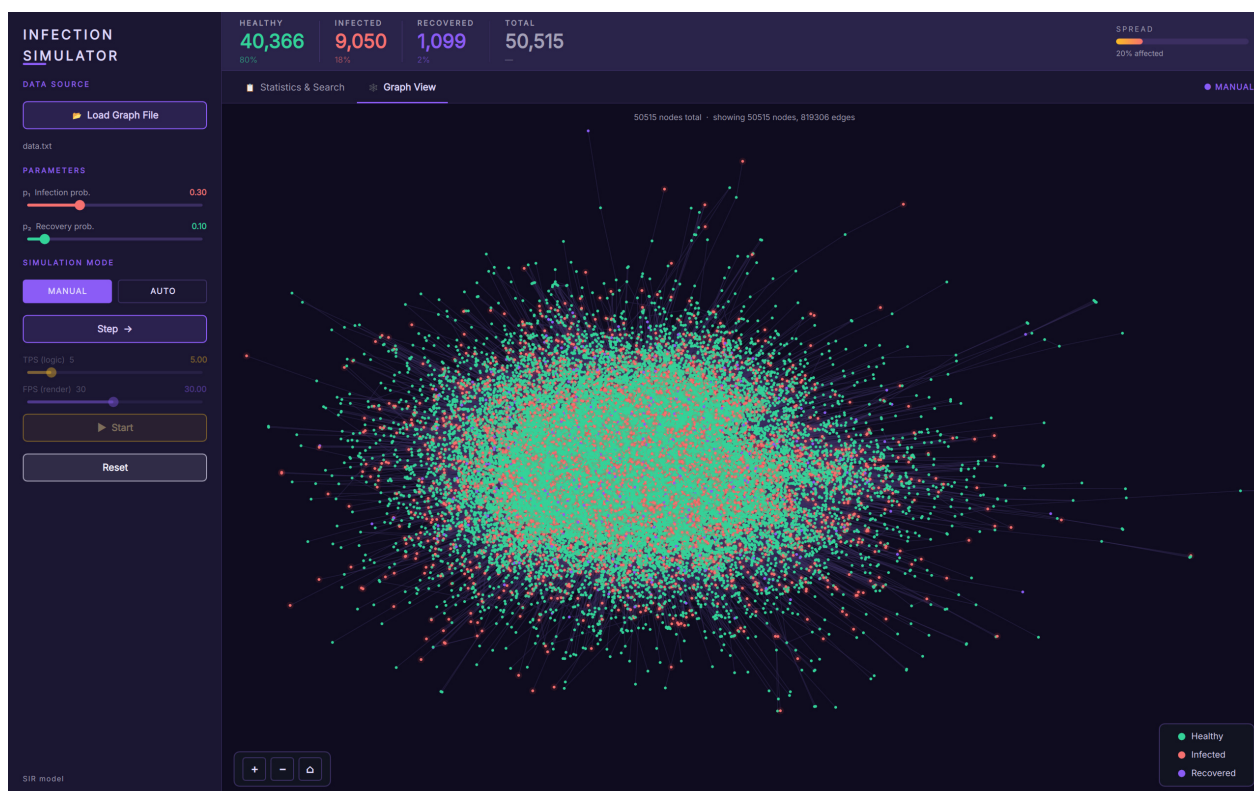
```

2.4 Тестовые данные и визуализация

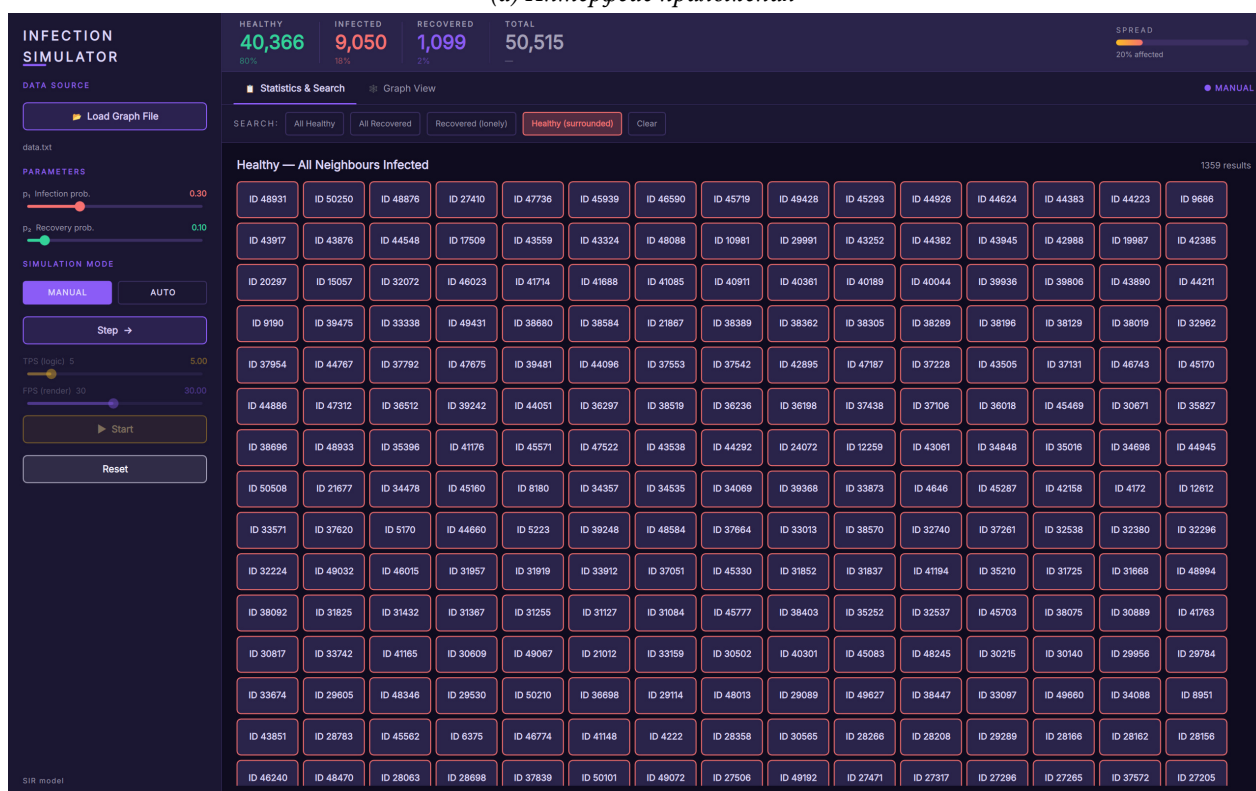
Тестовый граф — датасет Facebook-страниц категории «Artist» из коллекции GEMSEC. Исходный CSV-файл содержит рёбра в формате "*u, v*". Для загрузки выполняется предобработка: запятая заменяется пробелом. Параметры моделирования: $p_1 = 0.3$, $p_2 = 0.1$.

Визуализация графа реализована на QML с использованием **Canvas**. Для работы с большими графами (до 10^5 вершин) строится BFS-подграф от вершины с максимальной степенью, ограниченный заданным числом узлов. Размещение вершин — force-based алгоритм.

Отрисовка дифференцирует состояния вершин цветом: зелёный — Healthy, красный — Infected, синий — Recovered. Рёбра отображаются при достаточном масштабе. Поддерживаются панорамирование и масштабирование. Управление симуляцией и запросами осуществляется через боковую панель интерфейса.



(a) Интерфейс приложения



(b) Пример аналитического запроса

Рис. 2. Интерфейс приложения и пример аналитического запроса

3 Распространение слухов

В городе проживает $n + 1$ человек. Один из них узнаёт новость и передаёт её другому, тот — следующему, и так далее. На каждом шаге каждый знающий новость выбирает N случайных адресатов из n остальных жителей и сообщает новость им. Требуется определить вероятность того, что новость будет передана r раз без: а) возвращения к первому узнавшему (режим `ReturnToSender`); б) повторного сообщения кому-либо (режим `RepeatedMessage`). Необходимо реализовать приложение для моделирования процесса с настраиваемыми параметрами n, r, N, K (число экспериментов), поддерживающее пошаговый и пакетный режимы, визуализацию графа передачи и подсчёт эмпирических вероятностей.

3.1 Модель и алгоритм

Процесс моделируется на полном графе из $n + 1$ вершин. Вершина 0 — источник. Состояние системы — булев массив `knows` размера $n + 1$, изначально `knows[0] = true`. На шаге t каждый знающий выбирает N случайных получателей среди вершин, отличных от него самого.

3.2 Ядро симуляции

Listing 6: Метод `run()` — выполнение одного эксперимента

```
1 std::pair<bool, std::vector<Simulation::GraphNode>> Simulation::run(  
    size_t r, int seed) {  
2     m_rng = std::mt19937(seed);  
3     std::uniform_int_distribution<int> dist(0, m_n);  
4     std::vector<bool> knows(m_n + 1, false);  
5     knows[0] = true;  
6     std::vector<GraphNode> graph;  
7  
8     for (size_t step = 0; step < r; ++step) {  
9         std::vector<size_t> knowers;  
10        for (size_t i = 0; i <= m_n; ++i)  
11            if (knows[i]) knowers.push_back(i);  
12  
13        for (size_t sender : knowers) {  
14            for (size_t j = 0; j < m_N; ++j) {  
15                int target;  
16                do { target = dist(m_rng); } while (static_cast<size_t>(target) == sender);  
17            }
```

```

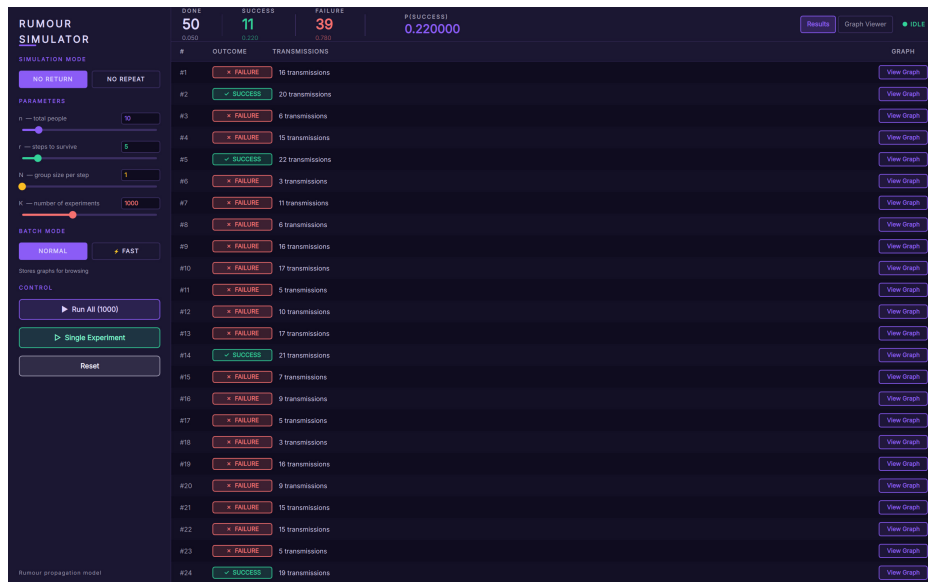
18     if (m_mode == Mode::ReturnToSender) {
19         if (target == 0) {
20             graph.push_back({sender, 0, step});
21             return {false, graph};
22         }
23     } else {
24         if (knows[static_cast<size_t>(target)]) {
25             graph.push_back({sender, static_cast<size_t>(target),
26                             step});
27             return {false, graph};
28         }
29         graph.push_back({sender, static_cast<size_t>(target), step});
30         knows[static_cast<size_t>(target)] = true;
31     }
32 }
33 }
34 return {true, graph};
35 }

```

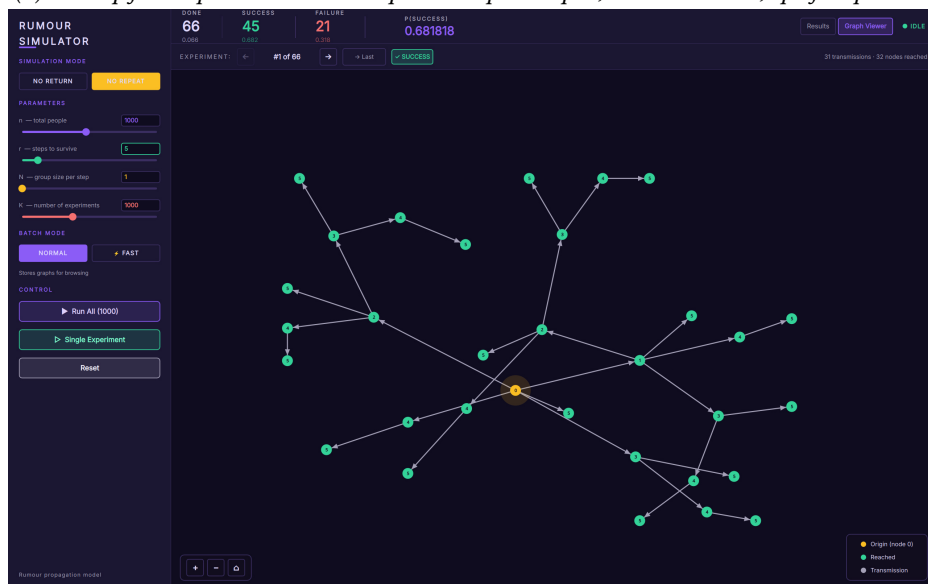
3.3 Визуализация и результаты

Граф передачи для каждого эксперимента отображается в компоненте `ExperimentGraphView`. Вершины размещаются силовым алгоритмом с анимацией сходимости.

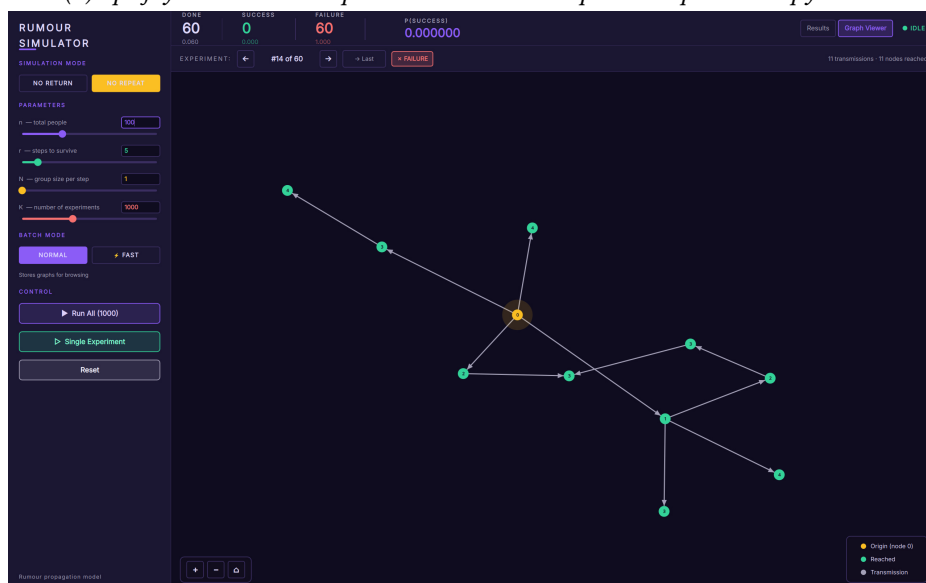
Интерфейс позволяет настраивать n , r , N , K , выбирать режим и запускать моделирование в пошаговом (с визуализацией каждого эксперимента) или быстром пакетном режиме. Статистика (число успехов/неудач, эмпирическая вероятность) обновляется в реальном времени.



(а) Интерфейс приложения: настройка параметров, статистика, граф передачи



(б) Граф успешного эксперимента: новость передана r раз без нарушений



(с) Граф неудачного эксперимента: нарушение условия (возврат к источнику / повтор)

Рис. 3. Интерфейс приложения и примеры результатов моделирования

4 Моделирование движения материальной точки со случайным шагом

h

5 Кластеры и k-паттерны

Здесь будет описание задачи и её решения.

6 Ступенчатая фигура

Здесь будет описание задачи и её решения.

7 Комплексное блуждание

Здесь будет описание задачи и её решения.

8 Бочка с бензином

Здесь будет описание задачи и её решения.

9 Криптоанализ

Здесь будет описание задачи и её решения.

10 Подбрасывание монеты из кошелька

Здесь будет описание задачи и её решения.

11 Дискретная случайная величина

Здесь будет описание задачи и её решения.

12 Игла Бюффона

Здесь будет описание задачи и её решения.

13 Статистика графа

Здесь будет описание задачи и её решения.

Список литературы

[1] Фамилия И.О. Название книги. — Город: Издательство, 2024. 256 с.